

Advanced Programming

CO2039

Chapter 5: Design Patterns

► CONTENT

- 01 INTRODUCTION TO DESIGN PATTERNS**
- 02 CREATIONAL PATTERNS - SINGLETON**
- 03 STRUCTURAL PATTERNS - ADAPTER**
- 04 BEHAVIORAL PATTERNS - OBSERVER**
- 05 CONCLUSION**



01

INTRODUCTION TO DESIGN PATTERNS



What is a design pattern?

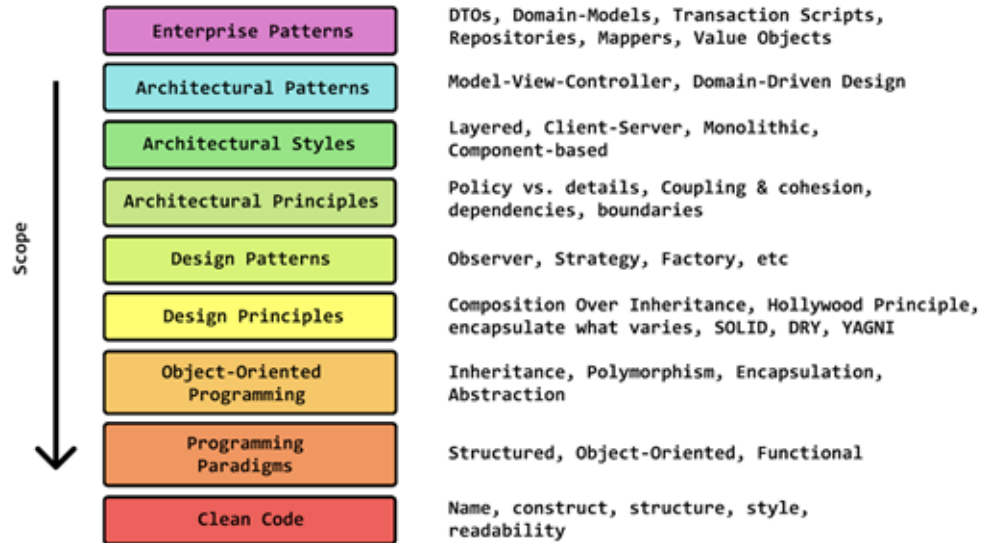
Design patterns are **typical solutions** to **commonly occurring problems** in **software design**.

They are like **pre-made blueprints** that you can **customize** to solve a recurring design problem in your code.



The Software Design & Architecture Stack

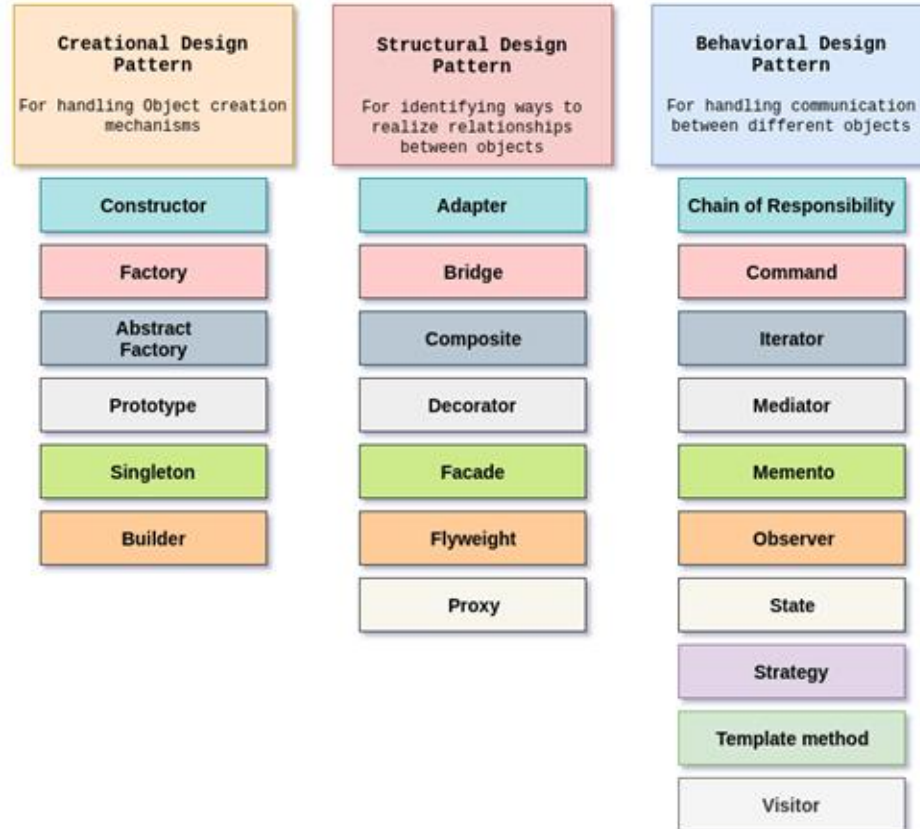
Khalil Stemmler
@stemmlerjs



Why should I learn patterns?

- Design patterns are a toolkit of **tried and tested solutions** to common problems in software design. Even if you never encounter these problems, knowing patterns is still useful because it teaches you **how to solve** all sorts of **problems using principles of object-oriented design**.
- Design patterns define a **common language** that you and your teammates can use to communicate more efficiently.
- Good designers **don't always start from scratch** to solve problems. They often **use existing solutions**!

Organization





02

CREATIONAL PATTERNS - SINGLETON



What is Singleton?

Singleton is a **creational design pattern** that lets you ensure that a **class has only one instance**, while **providing a global access point** to this instance.

WHY? Solve **2 problems** violating *Single Responsibility Principle*:

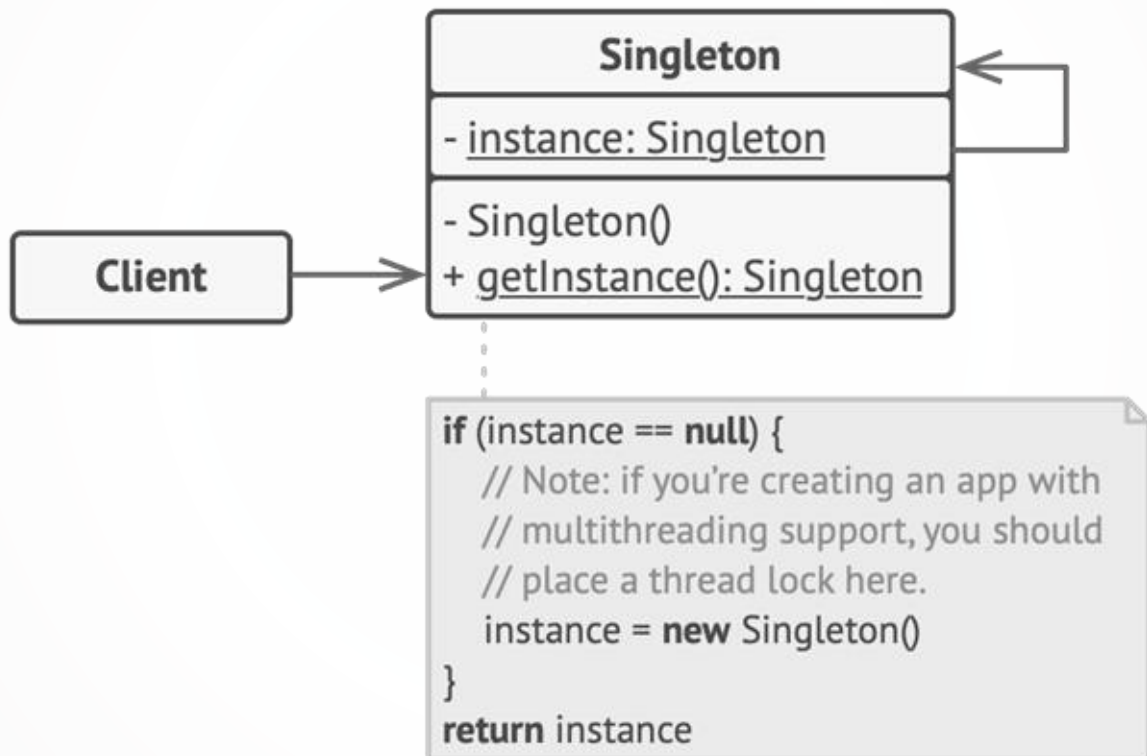
- **Ensure a single instance**
 - Prevent multiple instances of a class.
 - Control access to shared resources (e.g., database, files).
- **Provide global access**
 - Act like a global variable but prevent accidental overwrites.
 - Centralize instance management within one class.

Implementation

All implementations of the Singleton have these **two steps** in common:

- Make the **default constructor private**, to prevent other objects from using the `new` operator with the Singleton class.
- Create a **static creation method** that acts as a constructor. Under the hood, this method calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object.

Structure



Step-by-step instructions

1. **Add a private static field** to the class for storing the singleton instance.
2. **Declare a public static creation method** for getting the singleton instance.
3. **Implement “*lazy initialization*”** inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. **Make the constructor** of the class **private**. The static method of the class will still be able to call the constructor, but not the other objects.
5. Go over the client code and **replace all direct calls to the singleton’s constructor** with **calls to its static creation method**.



03

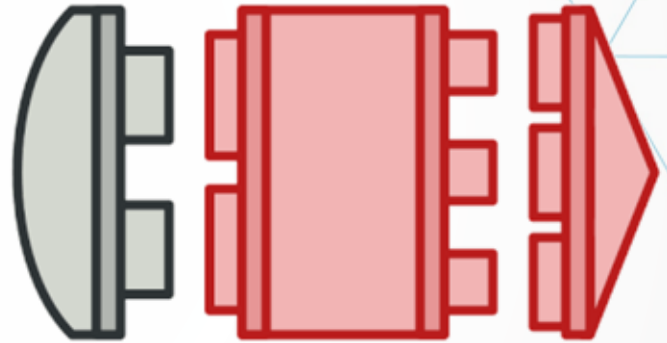
STRUCTURAL PATTERNS - ADAPTER

What is Adapter?

Adapter is a structural design pattern that allows objects with incompatible interfaces to collaborate.

WHY? Solve compatibility issue - bridge incompatible interfaces

- Allow two incompatible classes to work together.
- Convert one interface into another expected by the client.



Implementation

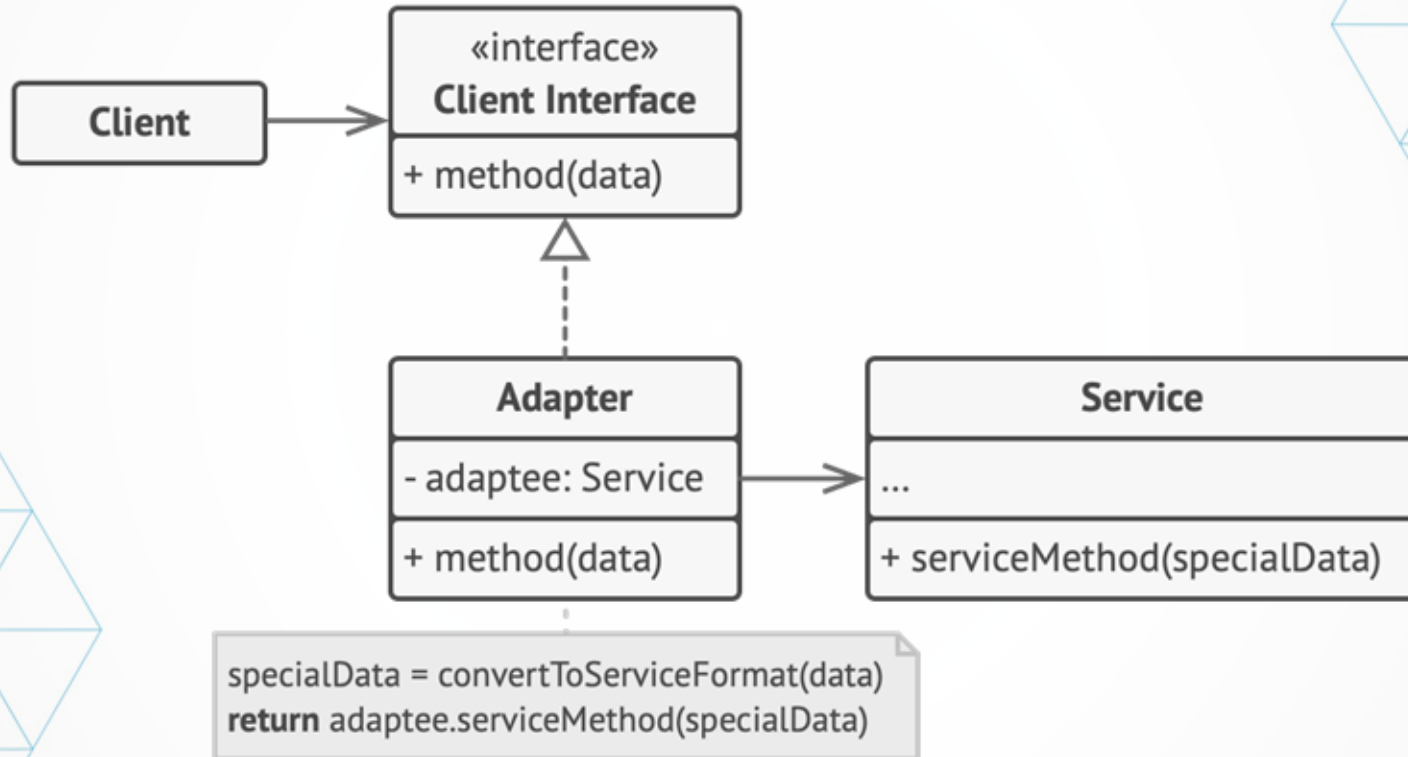
Hide complexity: Wrap an object to handle conversion behind the scenes.

Bridge interface: Allow incompatible objects to work together.

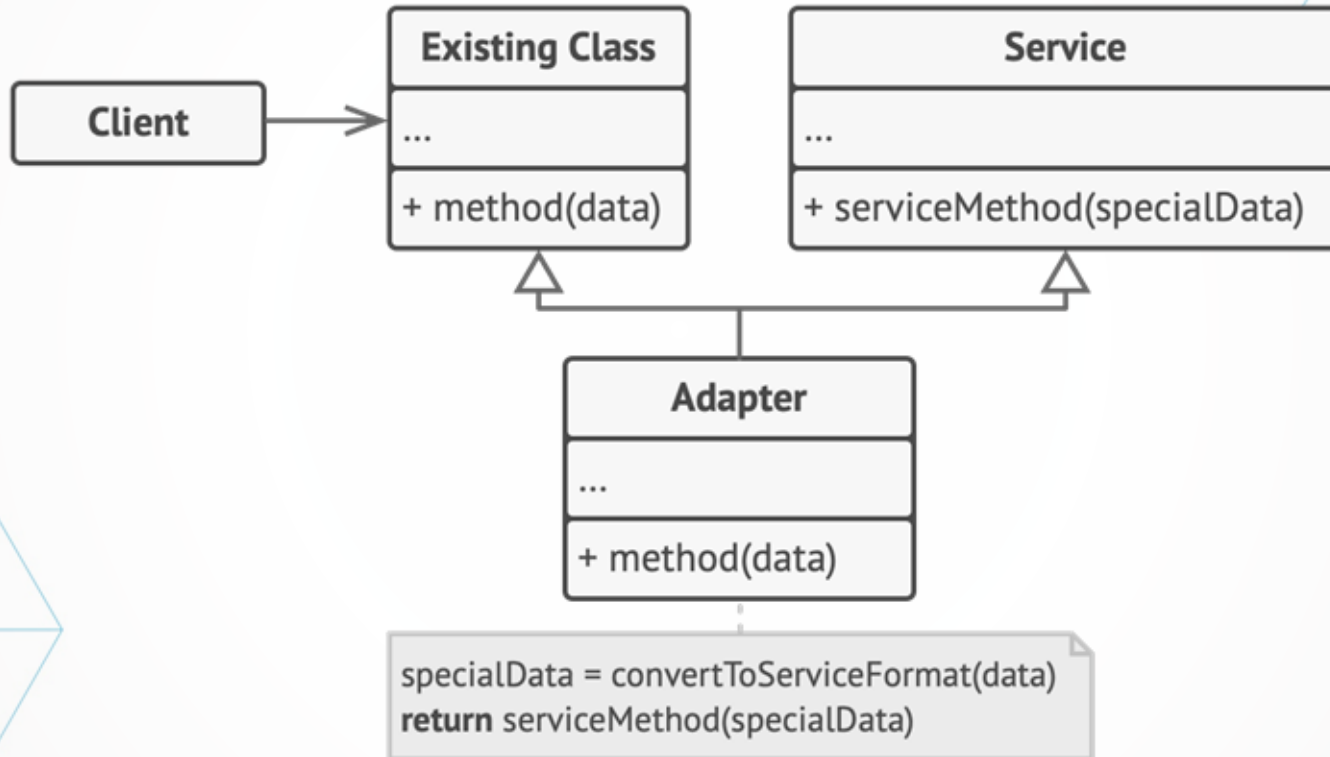
How it works:

1. Adapter provides an **interface** matching the client's needs.
2. Client **calls** the **adapter's methods**.
3. Adapter **translates the request** and **forwards it to the target object**.

Structure - Object adapter



Structure - Class adapter



Step-by-step instructions

1. Make sure that you have **at least two classes** with **incompatible interfaces**:
 - a. A useful service class, which you can't change.
 - b. One or several client classes that would benefit from using the service class.
2. **Declare the client interface** and describe how clients communicate with the service.
3. **Create the adapter class** and make it follow the client interface. Leave all the methods empty for now.
4. **Add a field to the adapter class** to store a reference to the service object.
5. One by one, implement **all methods** of the **client interface** in the **adapter class**.
6. Clients should **use the adapter via the client interface**. This will let you change or extend the adapters without affecting the client code.

Example: Sensor problem

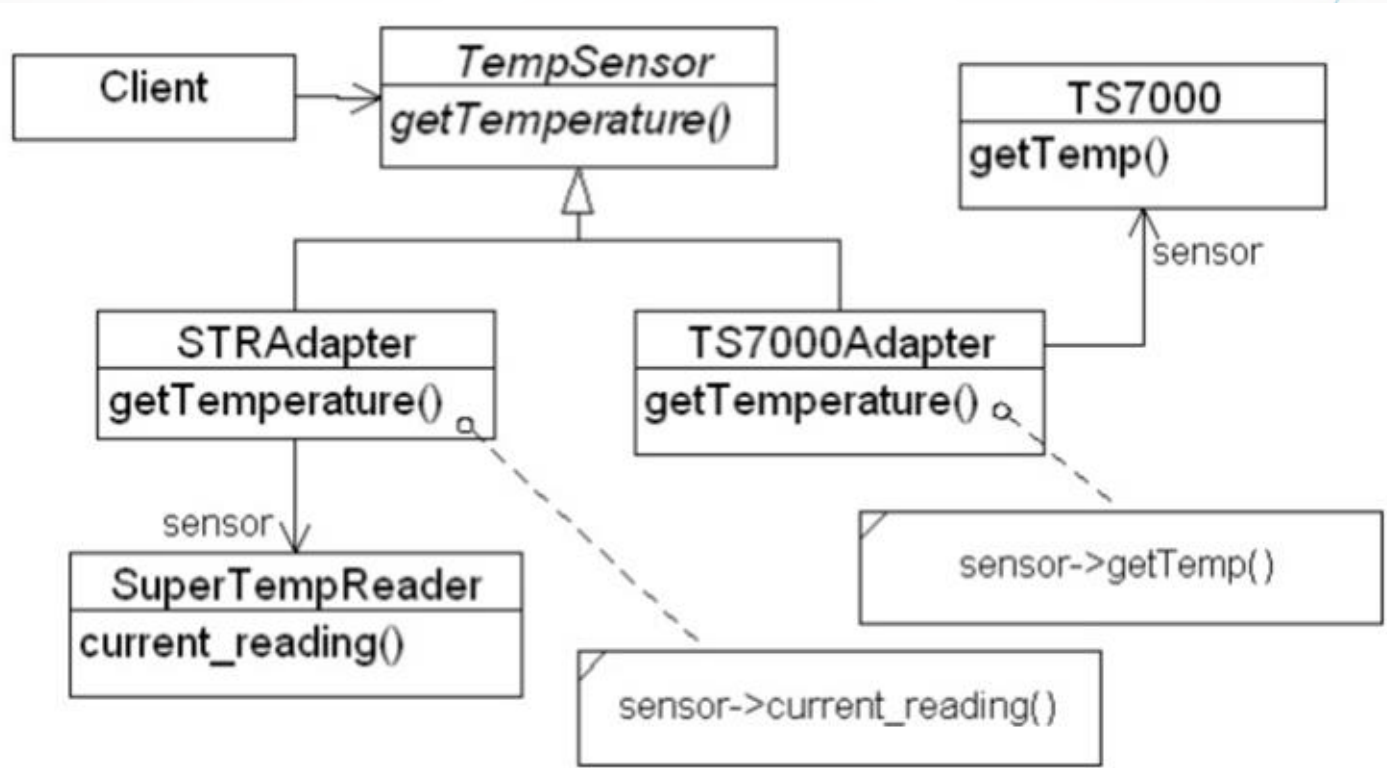
```
class TS7000 {  
    native double getTemp();  
    ...  
}  
  
...  
  
double sum = 0.0;  
for (int i = 0; i < sensors.length; i++)  
    sum += sensors[i].getTemp();  
double meanTemp = sum / sensors.length;
```

```
// New sensor device  
class SuperTempReader {  
    // NOTE: temperature is Celsius  
    tenths of a degree  
    native double current_reading();  
    ...  
}
```

Without adapter pattern

```
for (int i = 0; i < sensors.length; i++) {  
    if (sensors[i] instanceof TS7000)  
        sum += ((TS7000)sensors[i]).getTemp();  
    else  
        // Must be a SuperTemp!  
        sum += ((SuperTempReader)sensors[i]).current_reading() *  
            10;  
}
```

Adapter to the rescue (1)



Adapter to the rescue (2)

```
abstract class TempSensor{
    abstract double getTemperature();
}

class STRAdapter extends TempSensor{
    public double getTemperature(){
        return sensor.current_reading()
            * 10;
    }
}
```

```
class TS7000Adapter extends TempSensor{
    public double getTemperature(){
        return sensor.getTemp();
    }
}

...
double sum = 0.0;
for (int i = 0; i < sensors.length; i++)
    sum += sensors[i].getTemperature();
```



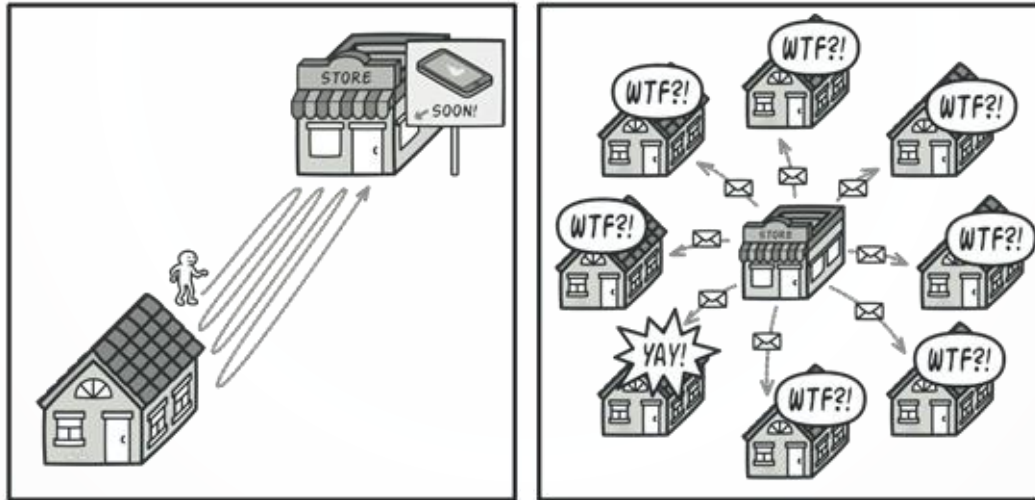
04

BEHAVIORAL PATTERNS - OBSERVER

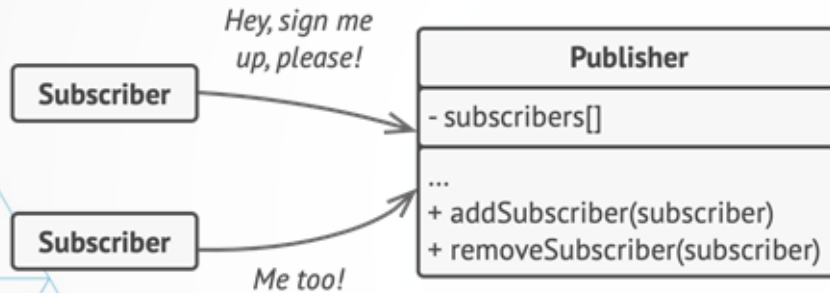


What is Observer?

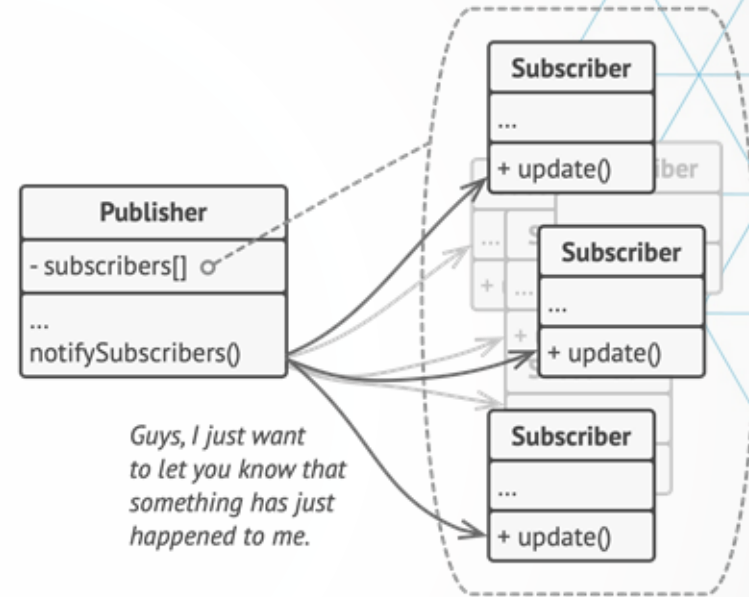
Observer is a behavioral design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing.



Implementation

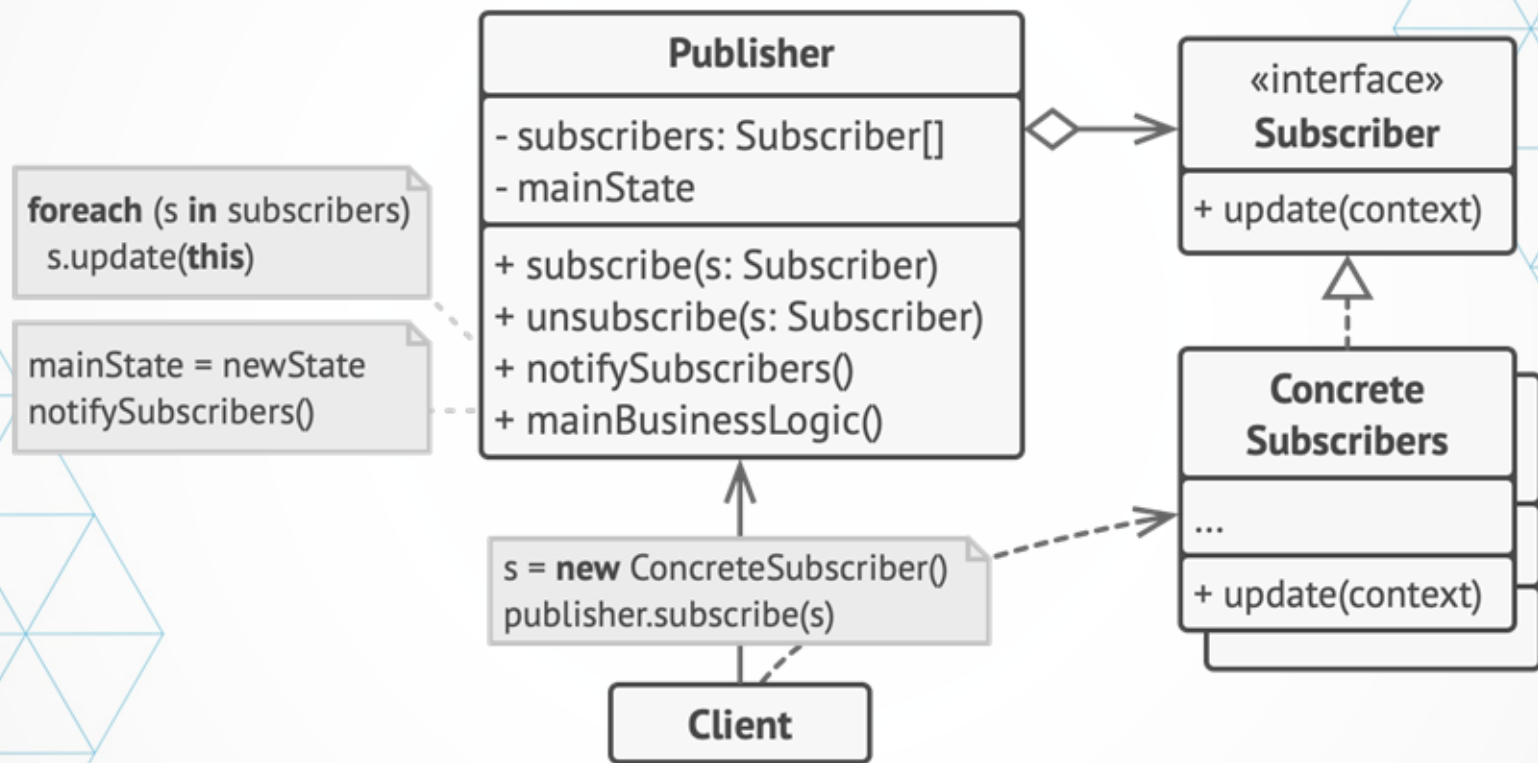


A subscription mechanism lets individual objects subscribe to event notifications.



Publisher notifies subscribers by calling the specific notification method on their objects.

Structure



Step-by-step instructions

1. Separate Business Logic

- Identify **core functionality** → Acts as the **Publisher**
- Identify **dependent components** → Act as **Subscribers**

1. Define Interfaces

- **Subscriber Interface** → Declares `update ()` method
- **Publisher Interface** → Methods to **add/remove** subscribers

1. Manage Subscriptions

- Store **subscriber list** in an **abstract class** (for reusability)
- Alternative: Use **composition** to separate subscription logic

1. Implement Concrete Classes

- **Publisher**: Notifies all **subscribers** when an event occurs
- **Subscriber**: Implements `update ()` method to react to changes

1. Register Subscribers: Clients create **subscribers** and attach them to **publishers**

05 CONCLUSION

Conclusion

- **Design Patterns** provide reusable solutions to common software design problems, improving code structure and maintainability.
- **Singleton** ensures a class has only one instance and provides global access, useful for managing shared resources.
- **Adapter** acts as a bridge between incompatible interfaces, enabling seamless integration of different systems.
- **Observer** establishes a publisher-subscriber model for dynamic event handling without tight coupling.

References

[1] Alexander Shvets (2018). *Dive Into Design Patterns*.

Thank you for your attention!

<https://www.cse.hcmut.edu.vn>